Streuspeichertechnik

Hashtables

Grundoperationen: - Löschen
- einfügen
- suchen
} O(1)

⇒ Schlüssel s wird vermöge einer Hash-Funktion ⇒ $n \in \mathbb{N} = h(s)$

Array:

Index	0	1	2	3	4
key	x_0	x_1	x_2	x_3	x_4
Data	y_0	y_1	y_2	y_3	y_4

(m-1) Einträge

Zugriff auf Einträge via key: h(key) ⇒ Index ⇒ A(Index) ⇒ Wert

Kollision

$h(s_1) = h(s_2)$ ⇒ referenziert auf gleiche Speicherstelle

Die Vermeidung bzw. der Umgang mit Kollisionen wird

über Interne und Externe Kollisionsauflösungsverfahren realisiert (s.u.)

Hashfunktion

Nimmt eine blibige Eingabemenge und bildet jedes Element auf einen bestimmten (zahlen) Wert ab.
⇒ Hashwert

Eigenschaften

(i) Deterministisch: Gleiche Eingabe liefert immer gleichen Hash
(ii) Effizient: Schnelle Berechnung, auch bei großer Eingabe
(iii) Nicht injektiv: Kollisionen möglich

Beispiel

Eingabe: String (Name)
h(s): Erster Buchstabe → ASCII-Wert mod X
X = 10

h("Alice") = 65 % 10 = 5
h("Bob") = 66 % 10 = 6
⋮

Kollisionsauflösung

Externe Kollisionsauflösung - Verkettung

Zusätzlich zu jedem Eintrag wird ein „Bucket" mitgeführt ⇒ Jeder Eintrag ist Kopfelement einer verketteten Liste
Im Falle einer Kollision werden Einträge mit gleichem Index zu einer solchen zusammengefügt
⇒ Je größer die Listen desto weiter geht die Laufzeit ges. O(n)

Beispiel

h(k) = (3k + 3) mod 8
Zahlenfolge: {12, 56, 47, 19, 89, 31, 24, 79}

Arr:

Index:	0	1	2	3	4	5	6	7
key:	0			3	4		6	7
Wert:	47			56	19		89	12

↓ 31 ↓ 79
↓ 24

Interne Kollisionsauflösung - Lineares Sondieren

Beim Auftritt eine Kollision wird via linearer Suche das nächst freie Feld ermittelt; $h_i(s)$: $h_0 = h(s)$, $h_1 = h(s)$ Ink(i,s), $i > 0$;
Inkrement ink(i,s) = c · i

⇒ kein zusätzlicher Speicherplatz, jedoch Clustering möglich

Beispiel

h(k) = (3k + 3) mod 8
Zahlenfolge: {12, 56, 47, 19, 89, 31, 24, 79}

Index:	0	1	2	3	4	5	6	7
Arr: key:	0	0	0	3	4	3	6	7
(Zahl, Anzahl Versuche einzufügen):	47 1	31 2	79 3	56 1	19 1	24 3	89 1	12 1

Cluster

Interne Kollisionsauflösung - Doppeltes Hashen

Verwendung von zwei Hashfunktionen h, g ab:

$ink(i,s) = g(s) \cdot i$ ⇒ Clustering minimiert

Um alle Tabelleneinträge zu durchlaufen muss gelten:

$ggT(h_1, m) = 1$ z.B. m Zweierpotenz & h_1 ungerade,

m Primzahl & $0 < h_1 < m$

Beispiel

$h(k,i) = (h_0(k) + i \cdot h_1(k)) \bmod 8$

$h_0(k) = (3k+2) \bmod 8$

$h_1(k) = 2 - (k \bmod 3) - 1$

Zahlenfolge: {12, 56, 47, 89, 31, 82, 10}

Index:	0	1	2	3	4	5	6	7
Arr: key:	0	1	2	3		5	6	7
(Zahl, Anzahl Versuche einzufügen):	31 2	82 0	56 1	10 4		89 1	12 1	47 1

82 → 10 → 10 → 10 → 31

$h_0(31) = 7 \overset{i=1}{\Rightarrow} (7 + 1 \cdot 1) \bmod 8 = \underline{0}$ ...

Interne Kollisionsauflösung - Quadratisches Sondieren

Gleiches Vorgehen beim Auftritt einer Kollision, jedoch wird das Inkrement quadratisch bestimmt:

$ink(i,s) = c \cdot i^2 + d \cdot i$

⇒ Funktioniert bestmöglich, wenn Länge(Arr) Primzahl ist

Beispiel

$h(k) = (3k+2) \bmod 12 + c_1 \cdot i + c_2 \cdot i^2$

$c_1 = 0{,}5$, $c_2 = 1{,}5$; i ist Zähler

Zahlenfolge: {12, 56, 47, 89, 31, 82, 10, 11}

Index:	0	1	2	3	4	5	6	7
Arr: key:	2	4	11	5	1	8	10	6
(Zahl, Anzahl Versuche einzufügen):	12 1	56 2	47 1	89 1	31 2	82 1	10 1	11 3

$h_0(12) = 2$, $h_0(56) = 2 \Rightarrow h_1(56) \overset{i=1}{=} 4$ ...

i	1	2	3
$c_1 i + c_2 i^2$	2	7	15